

Generalized Linear Models

Ashley I Naimi

October 2026

Contents

1	Introduction	2
2	Linear Versus Nonlinear Models	3
3	The Exponential Family: One Form, Many Models	7
3.1	Canonical Links and What They Represent	11
4	The GLM Menu	12
4.1	Gaussian	14
4.2	Binomial	15
4.3	Poisson	22
4.4	Negative Binomial	25
4.5	Gamma	29
4.6	Inverse Gaussian	30
4.7	One Mean Model but Many Distributions	32
5	Conclusion	36

1 Introduction

In Week 4 we illustrated the core ideas behind optimizing a regression model by minimizing least squares, and maximizing the likelihood function. Our least squares procedure found the line of best fit relating 1982 weight to 1971 weight, and we did this with nothing more than means and cross products (and the `lm()` returned exactly the same numbers). Maximum likelihood found the log odds ratio in a two by two table of infants by solving two score equations (the derivative of the log likelihood function), and `glm()` returned the same two numbers.

In Week 5 we conceptually decomposed a model into component parts, and discussed each in turn: the left hand side, the right hand side, the target and nuisance functions, the link, the distribution, and the offset. This week, we dive into the concept of “generalized linear models”, and explain why some link and distribution pairs are called “canonical”, and what, if anything, that word obliges us to do. We also explain why a binomial model with a log link so often fails to converge, and what to do about it.

One of the key messages in this lecture is that generalized linear models are a single family (i.e., members of the exponential family), and one algorithm is used to fit every member of this family (Nelder and Wedderburn, 1972; McCullagh and Nelder, 1989). Estimating the parameters of a logistic model, a Poisson model, a log-binomial model, a linear model, or other does not really require four methods. We use one algorithm with two functions plugged in: a variance function that says how the outcome scatters around its mean, and a link function that says on what scale the mean is additive in the covariates. The second key message is that we can often use this fact to our advantage and avoid the need to model the full distribution of an outcome variable. In fact, we can specify a mean function, through some link, and then separately specify a variance function around the mean, to estimate the parameters of a regression model that we might use to answer epidemiologic questions. In other words, we need not stick with the canonical link when using GLMs.

To start, we review the linear model and ask what least squares is minimizing and why, and we see that the answer is a likelihood. We then see why other members of the family (log-Poisson, logistic, gamma, and others) lose the closed form solution that characterizes the linear model, which means that we

What to read: Clark & Berry. Generalized Linear Models. Chapter 8 of *Models Demystified*: m-clark.github.io/book-of-models

can no longer come up with simple, single equations that represent solutions to the score function to estimate parameters. Instead, we need an iterative algorithm to estimate parameters, and we'll briefly cover how this works. The second half of the lecture will be applied, in that we'll fit the GLM model by model using a range of distributions and link function combinations, and run experiments showing that GLM coefficient estimates are not really unique to a specific distribution. We'll discuss why.



Note: Generalized Versus General

The abbreviation GLM is used for two different things. Less commonly (now, at least) the *general* linear model is the linear regression model with normally distributed errors, the model that unifies regression and analysis of variance. More commonly, the *generalized* linear model is the family introduced by Nelder and Wedderburn in 1972, of which the general linear model is one member. Older software called the latter GLIM, after the program that first fit it. In this course, as is now convention in science, GLM will always mean the generalized linear model.

2 Linear Versus Nonlinear Models

Let's again start with a linear regression model, say, for weight in 1982 as a function of weight in 1971:

$$E(Y | X) = \beta_0 + \beta_1 X$$

assuming $Y | X \sim \mathcal{N}(\mu = E(Y | X), \sigma^2)$. The link is the identity, so the mean is additive in the covariates on its own scale. The distribution is Gaussian, so the outcome scatters around its conditional mean with a constant variance σ^2 that does not depend on the mean. There is no offset. Nothing needs to be added to make this a GLM.

As we saw in Week 4, ordinary least squares minimizes the sum of squared distances between each outcome and its fitted mean, $\sum_i (y_i - \beta_0 - \beta_1 x_i)^2$. However, if we write down the log-likelihood of the data under the Gaussian model to use for maximum likelihood estimation, it turns out we are also minimizing least squares:

$$\ell(\beta_0, \beta_1, \sigma^2) = -\frac{n}{2} \log(2\pi\sigma^2) - \frac{1}{2\sigma^2} \sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_i)^2$$

The only place the regression coefficients appear is inside the sum of squares (with a negative sign). So, as it turns out, maximizing the likelihood over the coefficients is therefore the same as minimizing the sum of squares (Dobson, 2002, Section 6.2.2). This means that ordinary least squares is maximum likelihood under the Gaussian distribution.

The linear model like this one is characterized by an important special property that other GLMs lack. If we take the derivative of the sum of squares with respect to the linear model coefficients and set it to zero, we get what's called the "normal equations," $X'X\beta = X'y$, and because these are linear in β we can solve in one step: $\hat{\beta} = (X'X)^{-1}X'y$. In other words, there is a "closed-form solution" to the problem.¹

However, in settings where the score is not a linear function of the parameters, we need something more. Technically, a nonlinear model is a model where the first derivative of the expectation taken with respect to the parameters is itself a function of the parameters (Seber and Wild, 1989). For example,

$$E(Y | X) = \exp(\beta_0 + \beta_1 X)$$

is a nonlinear model, because its first derivative taken with respect to β_1 is still a function of β_1 (and of β_0). Recalling that the derivative of $\exp(u)$ taken with respect to u is $\exp(u)$, the chain rule gives:

$$\frac{dE(Y | X)}{d\beta_1} = \frac{d \exp(\beta_0 + \beta_1 X)}{d\beta_1} = X \exp(\beta_0 + \beta_1 X)$$

If we build the normal equation for β_1 , expand, and set it equal to zero, we get:

$$\sum_i x_i y_i \exp(\beta_0 + \beta_1 x_i) - \sum_i x_i \exp[2(\beta_0 + \beta_1 x_i)] = 0$$

Here, because the derivative is itself a function of β_1 , the parameter sits inside an exponential in every term, and each term carries a different x_i . There is no common factor to divide out, so we can't isolate β_1 on one side of the equation the way we could for the linear model.²

This simple example illustrates the technical distinction between linear and nonlinear models. Estimates are obtained by setting the score (the first derivative of the sum of squares, or of the log-likelihood) to zero and solving for the parameters. The derivative of the mean function appears as a factor inside the score, so when it still depends on the parameters (as it did for β_1

¹ In week 4, we computed this for the NHEFS weights by hand, arriving at an intercept of 8.007 and a slope of 0.924, and `lm()` results agreed exactly. The closed form exists because the score (i.e., the derivative of the log-likelihood), is a linear function of the parameters.

² Taking the log of both sides gives $\log E(Y | X) = \beta_0 + \beta_1 X$, which is linear in the parameters. That is the idea behind a link function, and it is the model we use to illustrate IRLS below. But it does not produce a closed form, because the log of an expectation is not the expectation of the log, so we cannot simply regress $\log y$ on x . The equation above is still the one that has to be solved.

above) the equations can no longer be solved by algebraic rearrangement, and finding an estimate requires an iterative approach. This is the complication introduced by nonlinear models.



Note: Nonlinear versus curvilinear models

Note that, at least mathematically, there is an important distinction between nonlinear and curvilinear models. The latter are linear models whose relationships can't be adequately captured by straight lines. These are easy to find solutions for, since their first derivatives are not functions of parameters. For instance, for a quadratic model such as:

$$E(Y | X) = \beta_0 + \beta_1 X + \beta_2 X^2$$

The first derivatives taken with respect to each parameter turn out to be:

$$\frac{dE(Y | X)}{d\beta_0} = 1$$

$$\frac{dE(Y | X)}{d\beta_1} = X$$

$$\frac{dE(Y | X)}{d\beta_2} = X^2$$

Thus, even though the regression function will not be a "straight line" on a plot, this model is still linear.

The models in this lecture are all built on a linear predictor, $x'_i\beta$, and yet all but one of them (Gaussian distribution, constant variance) lack a closed form, because the mean is a nonlinear function of the predictor (i.e., it's wrapped in a log, logistic, or some other nonlinear link function). So how do we proceed with parameter estimation?

There are three ways forward.

The first is to use a "grid search" method. Choose a range of plausible values for each coefficient, lay a grid of candidate values across those ranges, evaluate the log-likelihood at every point of the grid with the data held fixed, and keep the point with the largest value. The problem?: For one, the number of grid points multiplies exponentially with the number of coefficients, so a model with eleven coefficients would need 10^{11} evaluations of the log-likelihood at a very coarse ten candidate values per coefficient. Thus, this method is rarely deployed because of how burdensome and computationally demanding it is in realistic settings.

The second is a generic approach applied to a range of models, and quite common in machine learning. It is to use a numerical optimizer that climbs up the function until it finds the top. In the Clark and Berry book, section 8.5, they do this with the `optim()` function in R. This happens to be what gradient descent does in machine learning, and there are quite a few optimization algorithms out there (Newton-Raphson, BFGS, Nelder-Mead, Genetic Algorithms, Simulated Annealing).

As we'll see, running `optim()` on the logistic log-likelihood for the NHEFS model below recovers the same coefficients `glm()` reports.

The third is the trick that defines a generalized linear model. It's to notice that the score equations of a GLM have a specific structure that we can exploit to turn the nonlinear problem into a sequence of solvable weighted least squares problems. This procedure is referred to as "iteratively re-weighted least squares", which is, in fact, what `glm()` does. It's why `glm()` is fast (much faster than relying on something like `optim()`), and it is why its printout ends with a line about the number of "Fisher scoring" iterations.

We get to this idea of solving for GLM parameters iteratively with a small generalization of the closed form $\hat{\beta} = (X'X)^{-1}X'y$. In effect, we use a *weighted* least squares estimator of the form:

$$\hat{\beta} = (X'WX)^{-1}X'Wz$$

where W is a symmetric matrix with a diagonal of "weights", and where z is some mathematically updated version of the outcome used to find solutions to β even though the link functions / models used are not linear.

Getting into the specifics/math of IRLS is not worth our time here, but it is useful to at least build a general understanding of how the process works. Suppose I specify a log-linear model:

$$\log E(Y | X) = \beta_0 + \beta_1 X$$

where Y is assumed to follow a Poisson distribution with both a mean and variance $\lambda = \exp(\beta_0 + \beta_1 X)$. IRLS fits this model by starting with a "guess" for β_0 and β_1 , which gives every person a fitted mean $\hat{\lambda}_i = \exp(b_0 + b_1 x_i)$. From that guess, two quantities are calculated. The first is the "working response", $z_i = \log \hat{\lambda}_i + (y_i - \hat{\lambda}_i) / \hat{\lambda}_i$, which is the outcome mean for person i on the log scale **plus** the residual from this mean rescaled by how steeply the mean changes with the predictor at that point.

The second is the weight, $w_i = \hat{\lambda}_i$, the inverse of the variance of that working response, since dividing the Poisson variance $\hat{\lambda}_i$ by $\hat{\lambda}_i^2$ leaves $1/\hat{\lambda}_i$; people with larger expected counts carry more weight because their working response is more precise. Regressing z on X with the weighted least squares formula above returns a new set of coefficients. These new coefficients are then plugged back into the model, which changes every $\hat{\lambda}_i$, giving a new z and w , so the “re-weighted” least squares regression is run again. The loop keeps going until some criterion is met, for instance, that the change in the coefficients between iterations is sufficiently small (i.e., within some tolerance, such as 1×10^{-6}).

Why does this converge, and why to the maximum likelihood estimate? Think of the working response as the outcome seen through a straight-line approximation to the link. We cannot regress $\log y_i$ on x_i directly, because some counts are zero and because the log is curved, so instead we draw the tangent to the log at the current fitted mean and read each y_i off that tangent line: that is what $z_i = \log \hat{\lambda}_i + (y_i - \hat{\lambda}_i)/\hat{\lambda}_i$ is. If the current guess were exactly right, the z_i would scatter around the true line $\beta_0 + \beta_1 x_i$ with errors that average zero, and regressing them on x would return the true coefficients up to sampling error. When the guess is wrong, the tangent is drawn at the wrong place, so the z_i are slightly off, and the regression returns coefficients that are closer to the answer than the guess was but not yet equal to it.

The weights determine how much each z_i is trusted. Drawing the tangent stretches the noise in y_i by the slope of the link at $\hat{\lambda}_i$, and the noise in y_i itself depends on $\hat{\lambda}_i$ through the variance function. The weight undoes both, so that every observation counts in proportion to the precision of its working response. This makes the iteratively re-weighted least squares answer the maximum likelihood answer rather than merely an asymptotically consistent one.

Every member of the GLM family is fit by this same IRLS loop. The only things that change from one model to the next are the formulas for z and w , which depend on the link and on the variance function of the distribution.

3 The Exponential Family: One Form, Many Models

The loop in the previous section required a variance function $V(\mu)$ and the derivative of the mean with respect to the linear predictor. But it did not ask

for the distribution's density, its moment generating function, or anything else (which, as we saw in Week 4, is required for MLE to work). Why is that?

It turns out that every distribution that the GLM family admits can be written in one common form, referred to as the exponential family form. In that form the mean and the variance function are determined by a single function. There are several distributions that can be re-formatted to fit into the exponential family, and they include the Gaussian, the binomial, the Poisson, the gamma, and several others ([McCullagh and Nelder, 1989](#); [Dobson, 2002](#)).

A distribution belongs to the exponential family if its probability density or mass function can be written as

$$f(y; \theta, \phi) = \exp \left\{ \frac{y\theta - b(\theta)}{a(\phi)} + c(y, \phi) \right\}$$

where θ is called the natural or canonical parameter, ϕ is a dispersion parameter, and a , b , and c are known functions that differ from one member to the next. The function b carries all the information about the mean and the variance.



Note: How the GLM Distributions Fit into the Exponential Family

To see an example of how, we can look at how we can re-write the distributions for the Poisson model, the binomial model, and the Gaussian model. The Poisson probability function is $f(y; \mu) = \mu^y e^{-\mu} / y!$. Taking its logarithm and exponentiating back puts it in the form:

$$f(y; \mu) = \exp \{y \log \mu - \mu - \log y!\}$$

The term multiplying y is the natural parameter, $\theta = \log \mu$. The term in θ alone is $-b(\theta)$, with $b(\theta) = e^\theta$ since $\mu = e^\theta$. The dispersion is $a(\phi) = 1$, and what is left, $-\log y!$, is $c(y, \phi)$.

The binomial, written for a single Bernoulli trial with $\mu = P(Y = 1)$, is $f(y; \mu) = \mu^y (1 - \mu)^{1-y}$, which, after taking the log, and then re-exponentiating, gives

$$f(y; \mu) = \exp \left\{ y \log \frac{\mu}{1 - \mu} + \log(1 - \mu) \right\}$$

so the natural parameter is the log odds, $\theta = \log[\mu/(1 - \mu)]$. Solving for μ gives $\mu = e^\theta / (1 + e^\theta)$, so $\log(1 - \mu) = -\log(1 + e^\theta)$ and $b(\theta) = \log(1 + e^\theta)$, with $a(\phi) = 1$ and nothing left over for c .

The Gaussian, $f(y; \mu, \sigma^2) = (2\pi\sigma^2)^{-1/2} \exp\{-(y - \mu)^2/(2\sigma^2)\}$, needs the square expanded:

$$f(y; \mu, \sigma^2) = \exp \left\{ \frac{y\mu - \mu^2/2}{\sigma^2} - \frac{y^2}{2\sigma^2} - \frac{1}{2} \log(2\pi\sigma^2) \right\}$$

Now $\theta = \mu$, $b(\theta) = \theta^2/2$, the dispersion $a(\phi) = \sigma^2$ sits under the first term, and the rest is $c(y, \phi)$.

Two identities hold for every member of the exponential family, and these are:

$$E(Y) = b'(\theta), \quad \text{Var}(Y) = a(\phi) b''(\theta) = a(\phi) V(\mu)$$

The first says the mean function is the derivative of b . The second says the variance is the second derivative b , scaled by the dispersion, and because θ is a function of μ , the second derivative can be written as a function of the mean.

The point here is that the variance function is not an assumption we add on top of the distribution. Once the distribution is named, both the mean function and variance function (which is itself a function of the mean) follow from b .

We can apply the two identities to any of the distribution functions that are a part of the exponential family. For the Poisson, $b'(\theta) = b''(\theta) = e^\theta = \mu$: the mean and the variance are the same number. For the Bernoulli, $b'(\theta) = e^\theta / (1 + e^\theta) = \mu$ and $b''(\theta) = \mu(1 - \mu)$. For the Gaussian, $b'(\theta) = \mu$ and

$b''(\theta) = 1$, so the variance is σ^2 whatever the mean.

The following table shows these three and three more members of the exponential family.

Family	Natural parameter θ	$b(\theta)$	Canonical link	$V(\mu)$	Dispersion	R family
Gaussian	μ	$\theta^2/2$	identity	1	estimated	gaussian
Binomial	$\log[\mu/(1-\mu)]$	$\log(1+e^\theta)$	logit	$\mu(1-\mu)$	fixed at 1	binomial
Poisson	$\log \mu$	e^θ	log	μ	fixed at 1	poisson
Gamma	$-1/\mu$	$-\log(-\theta)$	inverse	μ^2	estimated	Gamma
Inverse Gaussian	$-1/(2\mu^2)$	$-\sqrt{-2\theta}$	$1/\mu^2$	μ^3	estimated	inverse.gaussian
Negative binomial	$\log[\mu/(\mu+\omega)]$	$-\omega \log(1-e^\theta)$	$\log[\mu/(\mu+\omega)]$	$\mu + \mu^2/\omega$	fixed at 1	negative.binomial

Six members of the exponential family. The natural parameter is the scale on which the distribution's own parameter enters the density linearly; the canonical link is the function that maps the mean onto that scale; the variance function is the second derivative of b . The last column is the name R uses in `family =`; the first five are in base R, and `negative.binomial()` comes from the MASS package. The negative binomial, sometimes used in epidemiology, belongs to the exponential family, but only when its shape parameter ω is held fixed.³

The unity the exponential family provides is not only mathematical. It is what makes `glm()` a single function. The `family` argument is a pointer to a row of the table above, an R object that carries the variance function, the link, its inverse, and the derivative the loop needs, and nothing else about the distribution. Here are three of the variance functions, printed as the family objects store them:

```
body(binomial())$variance)
```

```
## mu * (1 - mu)
```

```
body(poisson())$variance)
```

```
## mu
```

³ MASS: `glm.nb()` estimates ω outside the loop and pairs the family with the log link rather than its canonical one.

```
body(Gamma())$variance)
```

```
## mu^2
```

Everything the fitting routine does with a binomial family, it does by calling these functions inside the loop of the previous section. Swap the family object and the loop runs unchanged.

3.1 Canonical Links and What They Represent

The table above lists a canonical link for each family. Let's clarify what this word means. The natural parameter θ is a function of the mean. For the binomial it is $\log[\mu/(1 - \mu)]$, for the Poisson $\log \mu$, for the Gaussian μ itself. This natural parameter comes from taking the log, then exponentiating back, to transform the original distribution into the exponential family distribution. The canonical link is that function around the mean that results from this operation. A GLM uses the canonical link when the linear predictor is set equal to the natural parameter, $\eta_i = \theta_i$, so that the covariates enter the density on exactly the scale where the density is linear in its own parameter.

This has consequences for the machinery which we need not get into here.

Suffice it to say that the canonical link represents the scale on which the distribution's own parameter is additive in the covariates. For a Bernoulli outcome, the density is linear in the log odds, so a model that is linear in the log odds is a model that is linear in the natural parameter (and its coefficients are log odds ratios). This is why the logit is the default link for binary outcomes in every software package. It is not because odds ratios are the contrast an epidemiologist most often wants (in fact, they're not!).

The same goes for the Poisson and the log link, whose coefficients are log rate ratios, and for the gamma and the inverse link, whose coefficients are on a scale almost no one wants.



Note: Canonical Does Not Mean Correct

A common belief holds that fitting a GLM properly means choosing the distribution that matches the outcome and then the link that is canonical for that distribution, so that a binary outcome must get a logit. Nothing in GLM theory requires this. A binomial model with a log link, a binomial model with an identity link, and a Gaussian model with a log link are all generalized linear models, and the IRLS loop can fit each of them. What the non-canonical pairs, such as the log link with the binomial distribution, give up are some properties in the math that can matter. However, in practice these seem to not matter enough to make huge differences in the results we get, when we're appropriately cautious (e.g., when we correct the standard errors, when we ensure that fitted values don't lie too far outside the permissible range of the distribution, etc).

What we gain when we move away from the canonical link is a coefficient on the scale the question asks for (e.g., a risk ratio or a risk difference rather than an odds ratio). What they risk is a fitted mean outside its range, since the logit is the only one of the three binomial links that keeps every fitted probability between zero and one no matter what the coefficients are. The link chooses the contrast; the distribution chooses the variance function; the canonical pairing is the one case where the two choices happen to coincide.

There are certainly situations where choosing a non-canonical link can break the machine. For instance, if we use a log link for the binomial distribution, and note that some of the individual predicted probabilities are close to 2, this indicates a problem. For now, note that these problems tend to be rare in practice, and this experience is what is typically used to justify choosing non-canonical links in GLMs.

4 The GLM Menu

Now that we have a clearer understanding of the machinery behind a GLM, we can start discussing specific models.

The models epidemiologists fit most often are a menu of choices for main categories: the family, which provides us with the variance function, and the link, which sets the scale on which the mean is additive in the covariates.

In this section, we cover six families encountered in the table above. For each, we look at the distribution itself, simulate an outcome whose mean depends on a single covariate through the family's usual link, and fit the GLM to the simulated data to watch how the specific parameters we used to simulate the data return in the estimates. We also use the NHEFS data as an example where real data can show us something simulations can't. These include the log-binomial model's convergence failure, separation and quasi-complete separation, and the experiment at the end of the section, which fits one mean

model under several families. Week 5 reported the crude associations between quitting and above-median weight gain, a risk difference of 0.129, a risk ratio of 1.28, and an odds ratio of 1.68 from the three binomial links; the adjusted model adds age, sex, race, exercise, marital status, and smoking intensity.

The data are the Week 5 data. As in Week 5, the outcome is an indicator of weight change above the median, the exposure is quitting smoking, and marital status is recoded so that the separated are pooled with the divorced and the one person with unknown status joins the “married” group. We keep the raw coding as well, we’ll need it later in the lecture.

```
nhefs <- read_csv(here("data", "nhefs.csv")) %>%
  select(qsmk, wt82_71, wt82, wt71, exercise, sex, age, race,
         marital, smokeintensity, smkintensity82_71,
         cholesterol, death, yrdth) %>%
  filter(!is.na(wt82_71)) %>%
  mutate(wt_delta = as.numeric(wt82_71 > median(wt82_71)),
         marital_raw = marital,
         marital = ifelse(marital == 6, 5, marital),
         marital = ifelse(marital == 8, 2, marital))

f_adj <- wt_delta ~ qsmk + age + sex + race + factor(exercise) +
  factor(marital) + smokeintensity

grp <- nehs %>%
  group_by(qsmk) %>%
  summarise(n = n(), events = sum(wt_delta))
```

```
grp
```

```
## # A tibble: 2 x 3
##   qsmk      n events
##   <dbl> <int> <dbl>
## 1     0  1163    543
## 2     1   403    240
```

4.1 Gaussian

We can simulate data from a Gaussian distribution using the `rnorm` function in R. The Gaussian distribution is fully defined by its mean and standard deviation, and the default values for these in R are 0 and 1, respectively:

```
set.seed(123)
```

```
n <- 5
```

```
y <- rnorm(n, mean = 0, sd = 1)
```

```
y
```

```
## [1] -0.56047565 -0.23017749  1.55870831  0.07050839
```

```
## [5]  0.12928774
```

In a regression context, the mean of a Gaussian random variable is often what is made conditional on other variables. For example, in a simple setting, we might have a Gaussian outcome Y with a mean conditional on X . We can simulate this with a sample size of five thousand, and we see `glm()` returns exactly those parameters we used to simulate:

```
set.seed(123)
```

```
n <- 5000
```

```
x <- rnorm(n)
```

```
y <- 1 + 2*x + rnorm(n, mean = 0, sd = 1)
```

```
fit_gauss <- glm(y ~ x, family = gaussian("identity"))
```

```
summary(fit_gauss)
```

```
##
```

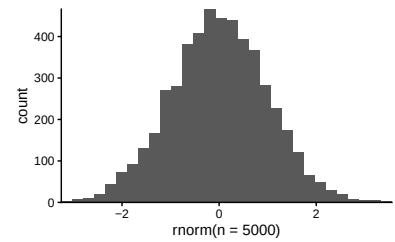


Figure 1: Histogram for the Gaussian distribution with mean 0 and standard deviation 1 for 5000 simulated observations.

```
## Call:
## glm(formula = y ~ x, family = gaussian("identity"))
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept)  0.99582    0.01418   70.21  <2e-16 ***
## x            1.99397    0.01426  139.81  <2e-16 ***
## ---
## Signif. codes:
## 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for gaussian family taken to be 1.005737)
##
##      Null deviance: 24687.1  on 4999  degrees of freedom
## Residual deviance:  5026.7  on 4998  degrees of freedom
## AIC: 14222
##
## Number of Fisher Scoring iterations: 2
```

The intercept and slope come back as 1 and 2 (to within sampling error), and the estimated dispersion is the error variance we simulated, one. The slope is a difference in means: each unit increase in x raises the expected outcome by 2. This is the one member of the family whose coefficients could have been found in closed form, so the IRLS loop converges on its first pass and stops on its second.

4.2 Binomial

The binomial distribution is defined by two parameters: the probability of a success in a given “trial” and the number of trials conducted. When the number of “trials” is one, the binomial distribution is equivalent to the Bernoulli distribution, which generates a $\{0, 1\}$ indicator of whether an event occurred or not. In settings where logistic regression is used, the dependent variable is usually from a Bernoulli distribution with a probability depending on variables that will be included in the model. We can define a linear term for a logistic regression model, define the inverse logistic link function, and then use these to

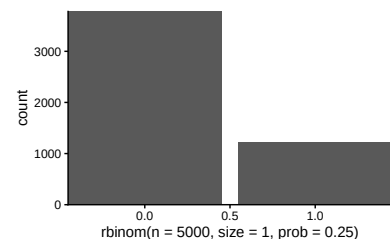


Figure 2: Barplot for the binomial (Bernoulli) distribution with $p = 0.25$ for 5000 simulated observations.

generate a Bernoulli outcome Y that depends on a single covariate C through the link function. With five thousand observations, the logistic model fit to the simulated data returns the parameters we put in:

```
set.seed(123)

n <- 5000

c <- rnorm(n)

expit <- function(x){1/(1+exp(-x))} # note this is equivalent to plogis() in base R

y <- rbinom(n, size = 1, prob = expit(-2 + log(2)*c))

fit_logit <- glm(y ~ c, family = binomial("logit"))

summary(fit_logit)
```

```
##
## Call:
## glm(formula = y ~ c, family = binomial("logit"))
##
## Coefficients:
##             Estimate Std. Error z value Pr(>|z|)
## (Intercept) -2.05305    0.04783  -42.92  <2e-16 ***
## c           0.69811    0.04582   15.23  <2e-16 ***
## ---
## Signif. codes:
## 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##    Null deviance: 3897.9  on 4999  degrees of freedom
## Residual deviance: 3644.1  on 4998  degrees of freedom
## AIC: 3648.1
##
```

```
## Number of Fisher Scoring iterations: 5
```

The intercept comes back near -2 and the slope near $\log(2) = 0.69$, so the odds ratio per unit increase in C is 2, which is what the simulation specified. The binomial family with the logit link is the canonical pair for a binary outcome, so this model's log-likelihood has one maximum, the loop converges from any starting point when a maximum exists, and no fitted probability can lie beyond the unit interval.

Earlier, we noted that a generic numerical optimizer would find the same coefficients that `glm()` reports. Here is that check on the adjusted NHEFS logistic model. We write the Bernoulli log-likelihood as a function of the coefficients, hand its negative to `optim()` together with its gradient (which is the score), and start the search at zero:

```
X <- model.matrix(f_adj, data = nhefs)
y <- nhefs$wt_delta

negloglik <- function(beta) {
  eta <- X %*% beta
  -sum(y * eta - log(1 + exp(eta)))
}

score <- function(beta) {
  eta <- X %*% beta
  -t(X) %*% (y - plogis(eta))
}

opt <- optim(par = rep(0, ncol(X)), fn = negloglik, gr = score, method = "BFGS")

mod_logit <- glm(f_adj, data = nhefs, family = binomial("logit"))

round(cbind(optim = opt$par, glm = coef(mod_logit)), 4)
```

```
##              optim      glm
## (Intercept)  1.5071  1.5071
## qsmk         0.6743  0.6743
```

```
## age          -0.0367 -0.0367
## sex          0.0496  0.0496
## race        -0.1918 -0.1918
## factor(exercise)1 -0.0850 -0.0850
## factor(exercise)2 -0.2315 -0.2315
## factor(marital)3  0.2606  0.2606
## factor(marital)4 -0.0114 -0.0114
## factor(marital)5 -0.0602 -0.0602
## smokeintensity  0.0018  0.0018
```

```
opt$counts
```

```
## function gradient
##          69          16
```

```
mod_logit$iter
```

```
## [1] 4
```

The two columns agree to four decimals in every row, because both routines are climbing the same log-likelihood and it has one peak. The difference is the work it took to get there: `optim()` evaluated the log-likelihood 69 times and its gradient 16 times, while the IRLS loop inside `glm()` needed four updates. Left to its default Nelder-Mead search with no gradient, `optim()` stops after 500 evaluations without converging. That gap is the practical reason the IRLS loop exists.

The same family with a log link returns a risk ratio instead of an odds ratio. It's the first place where the IRLS loop can fail. In the NHEFS data, we can see this. The adjusted model for above-median weight gain, with the Week 5 confounders, fails outright:

```
mod_logbin <- glm(f_adj, data = nhefs, family = binomial("log"))
```

```
## Error:
```

```
## ! no valid set of coefficients has been found: please supply starting values
```

Under the log link, the fitted probability is $\exp(x_i'\beta)$, which is bounded below by zero but not above by one. The binomial log-likelihood is undefined wherever a fitted probability exceeds one. Here, `glm()` starts the loop at coefficient values that put at least one person's predicted probability beyond the upper boundary. The routine cannot evaluate the likelihood there, so it stops and asks for a starting point inside the parameter space.

Generally, when this problem occurs, providing a starting value isn't the best practice and often doesn't work. Here, it does work. Starting the intercept at the log of the marginal risk and every other coefficient at zero puts every fitted probability at the overall risk, which is inside the range, so we end up getting an answer from this model:

```
mod_logbin <- glm(f_adj, data = nhefs, family = binomial("log"),
                 start = c(log(mean(nhefs$wt_delta)), rep(0, 10)))

round(exp(coef(mod_logbin))["qsmk"], 3)
```

```
## qsmk
## 1.333
```

```
mod_logbin$iter
```

```
## [1] 8
```

```
round(max(fitted(mod_logbin)), 3)
```

```
## [1] 0.85
```

The IRLS loop makes 8 updates, yields a risk ratio of 1.33, and a largest fitted probability of 0.85. Note here that providing starting values is what rescues this fit, and so it would be important to evaluate how sensitive these results are to the selected starting values, should we choose to use them in a real project. The rescued answer can depend on where the loop started, because the log-likelihood of a non-canonical model is not guaranteed to have a single peak. A fit that has been coaxed into converging near the edge of its parameter space should be interpreted knowing that a slightly different

sample could push it over. There are dedicated algorithms for exactly this type of model and problem, and these can be useful when this problem arises ([Marschner and Gillett, 2012](#)). However, they are not the routine solutions to this problem, which include using a Poisson family with the same log link and a robust variance (good), or a logistic model with the risk ratio computed from its predicted probabilities by standardization (better).


Note: Separation, Found in the Data

The log-binomial model fails because its fitted mean leaves its range. There is a second way a GLM fails to converge, in which a coefficient leaves the real line (i.e., starts to approach infinity). The NHEFS data contain an example of this. The raw marital status variable has six codes, and one of them, unknown, is held by a single person. Fit the adjusted logistic model with marital status as a factor on its raw coding:

```
mod_raw <- glm(wt_delta ~ qsmk + age + sex + race + factor(exercise) +
               factor(marital_raw) + smokeintensity,
               data = nhefs, family = binomial("logit"))

round(summary(mod_raw)$coefficients["factor(marital_raw)8", 1:2], 2)
```

```
##      Estimate Std. Error
##      11.35      324.74
```

```
mod_raw$iter
```

```
## [1] 11
```

```
round(fitted(mod_raw)[nhefs$marital_raw == 8], 6)
```

```
##      1270
## 0.999997
```

The coefficient for the unknown category is 11.35 with a standard error of 325, the loop needed eleven updates, and the one person in that category has a fitted probability of 0.999997. The “unknown” marital status person had the outcome, and a category that contains only them can be fit perfectly by sending its coefficient toward infinity (i.e., guaranteeing that the probability will be one). The likelihood keeps increasing along the way, so there is no maximum to find, and the loop stops only because the changes in the deviance eventually fall below its tolerance. This is called quasi-complete separation, because the individual(s) defined by one attribute in the model (here, marital status = “unknown”) all have the outcome. In other words, we can completely separate these individuals from the rest because there’s no variability in the outcome. Any category or combination of covariates that perfectly predicts the outcome for the people in it produces the same property: a very large coefficient, a larger standard error, and a long iteration count. In Week 5’s recode, which folded the unknown into the married, we removed the problem before it could appear. When recoding is not available, the general remedy is a penalty on the likelihood that keeps every coefficient finite ([Firth, 1993](#)), which we cover in Week 10.

4.3 Poisson

The Poisson distribution is fully defined by a single parameter, usually denoted λ . This distribution is usually used to model counts of discrete events, such as the number of cigarettes smoked in one hour among smokers. In a Poisson regression model, the log of λ is linear in the covariates, so to simulate from the model we exponentiate a linear term and hand the result to `rpois()` as the rate:

```
set.seed(123)

n <- 5000

x <- rnorm(n)

y <- rpois(n, lambda = exp(0.5 + log(1.5)*x))

fit_pois <- glm(y ~ x, family = poisson("log"))

round(coef(fit_pois), 3)
```

```
## (Intercept)      x
##      0.490      0.412
```

```
round(exp(coef(fit_pois))["x"], 3)
```

```
##      x
## 1.51
```

```
round(sum(residuals(fit_pois, type = "pearson")^2) / df.residual(fit_pois), 2)
```

```
## [1] 0.98
```

The slope returns $\log(1.5)$ and its exponential is the ratio of expected counts we simulated: each unit of x multiplies the expected count by 1.5. The last line is the Pearson statistic divided by its degrees of freedom, an estimate

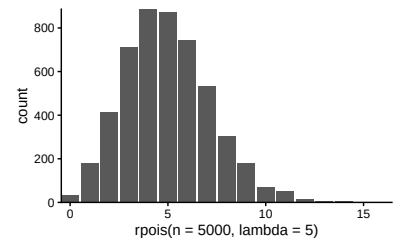


Figure 3: Barplot for the Poisson distribution with $\lambda = 5$ for 5000 simulated observations.

of how far the variance departs from the mean. In this case, it's close to one, as it should be when the data really are Poisson.

If we paired this model with a log person-time offset, we get expected rates instead of counts, and the coefficients become log rate ratios. We can simulate a cohort in which a binary exposure doubles the event rate, but in which exposed people are also followed for less than half as long as unexposed people. We then fit a Poisson model to the counts alone, and the same model with the offset:

```
set.seed(123)

n <- 5000

x <- rbinom(n, 1, 0.5)

t <- ifelse(x == 1, runif(n, 0.5, 2.5), runif(n, 2, 5))

y <- rpois(n, lambda = exp(log(0.2) + log(2)*x) * t)

fit_count <- glm(y ~ x, family = poisson("log"))
fit_rate <- glm(y ~ x + offset(log(t)), family = poisson("log"))

round(exp(coef(fit_count))["x"], 3)

##      x
## 0.803

round(exp(coef(fit_rate)), 3)

## (Intercept)      x
##      0.200      1.906

rate_1 <- sum(y[x == 1]) / sum(t[x == 1])
rate_0 <- sum(y[x == 0]) / sum(t[x == 0])
```

```
round(rate_1 / rate_0, 3)
```

```
## [1] 1.906
```

The model fit to the counts alone returns a ratio of 0.80, which says exposure is protective. It compares the number of events in each group with no regard for how much person-time each person contributed. The offset model returns 1.91, close to the rate ratio of 2 we simulated, and its exponentiated intercept is 0.20, the baseline rate per unit of person-time we simulated. The last line computes the crude incidence rate ratio by hand, total events over total person-time in each group, and it equals the offset model's estimate exactly.

When paired with a binary outcome, the Poisson distribution with log link becomes a device for estimating risk ratios ([Zou, 2004](#)). The adjusted NHEFS model that failed under the binomial family with a log link can be fit with the Poisson family and the same link, without starting values:

```
mod_pois <- glm(f_adj, data = nhefs, family = poisson("log"))
```

```
round(exp(coef(mod_pois))["qsmk"], 3)
```

```
## qsmk
```

```
## 1.355
```

```
mod_pois$iter
```

```
## [1] 5
```

```
round(max(fitted(mod_pois)), 3)
```

```
## [1] 0.9
```

The loop converges in five updates with no help, and returns a risk ratio of 1.36 (compared to 1.33 from the “starting values” log-binomial model). This fit does not need starting values because the log link is the Poisson's canonical link, which means that the fitted mean $\exp(x_i'\beta)$ can be any positive number

(Poisson distribution's full range), so the likelihood is defined everywhere and has a single peak. Nothing stops a fitted value from exceeding one in this case, so the largest fitted value (here 0.90) is worth checking whenever this route is taken.

The binomial distribution and the Poisson distribution GLMs weight the observations differently once the model is no longer saturated. This is why the two risk ratios differ slightly. Additionally, the Poisson's model-based standard errors are wrong for a binary outcome, since its variance function for the Poisson is μ , whereas the truth for this outcome is $\mu(1 - \mu)$.

In the experiment at the end of this section, we see both of these issues at play, and in Week 7 we show how the robust variance estimator actually corrects the variance problem.

4.4 Negative Binomial

The negative binomial distribution is commonly used in epidemiology, particularly when we have a count outcome, but where the variance of this count outcome is not necessarily equal to the mean (as Poisson assumes). There are a few ways in which one can represent a negative binomial distribution. The first is in relation to the standard binomial. This representation is sometimes referred to as the Pascal distribution (e.g., [Leemis et al., 2008](#)) or the “counting failures” version of the negative binomial distribution. This version is related to the binomial distribution in complement. That is, the binomial distribution can be used to model the number of successes X we'd expect to see for a scenario where there are a fixed number of trials N and a fixed probability of success p .⁴ In contrast, a negative binomial distribution can be used to model the **number of failures** Y we'd expect to see before we encounter a fixed number of successes r and a fixed probability of success p .

In R, we can simulate a random variable from a negative binomial distribution using this “counting failures” formulation as follows:

```
set.seed(123)

y <- rnbino(n = 5, prob = .2, size = 2)
```

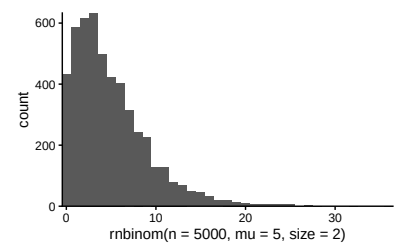


Figure 4: Barplot for the negative binomial distribution with mean 5 and size 2 for 5000 simulated observations.

⁴ In the Bernoulli case, the number of trials is 1, with a probability of observing a success defined by p .

```
y
```

```
## [1] 6 16 11 3 7
```

This tells us that we observed six failures before the first two successes, 16 failures before the next two successes, then eleven, then three, then seven. In other words, these represent the number of failures before we see two successes (since `size = 2`).

However, there is another way to formulate the negative binomial distribution, as a mixture of the Poisson and the Gamma distribution ([Casella and Berger, 2002](#), p. 196, exercise 4.34). This formulation allows us to think of the negative binomial distribution as a Poisson distribution, but without the assumption that the variance of this distribution is exactly equal to the mean. This is the most common formulation used in epidemiology (see, e.g., [Lloyd-Smith et al., 2005](#)). In R, this formulation can be simulated as:

```
set.seed(123)
```

```
y <- rnbinom(n = 5, mu = 5, size = 2)
```

```
y
```

```
## [1] 4 10 8 1 4
```

In this formulation, the variance of the negative binomial random variable is defined as:

$$\text{Var}(Y) = \mu + \frac{\mu^2}{\text{size}},$$

so for small values of `size`, we generate data that has a variance greater than the mean. As we increase the `size` parameter, we decrease the variance. In the limit as $\text{size} \rightarrow \infty$, this negative binomial distribution becomes a Poisson distribution ([Cameron and Trivedi, 2013](#), p. 85). R's `size` argument is ω in the table above, which is the shape parameter that the exponential family form holds fixed.

**Note:** Overdispersion

Overdispersion describes a situation where the variance in an outcome variable exceeds the variance assumed under a model's distributional assumptions. This concept is most commonly encountered with a Poisson random variable. The variance of a Poisson distribution is equal to the mean. If we let μ denote the mean of an outcome variable Y , and we let σ^2 denote its variance, then modeling Y with a Poisson regression model implies that $\mu = \sigma^2$. However, in actuality for some variables **assumed** to follow a Poisson distribution, we may find that $\mu < \sigma^2$. In this case, we would say that the outcome is “overdispersed”. One solution to overdispersion is to estimate a scale parameter in the GLM. In a Poisson regression context, adding a scale parameter re-defines the variance of the outcome to be $V(Y) = \phi\mu$, where ϕ is a scale parameter that multiplies this variance accordingly. We can set the scale parameter ϕ to 1 if there is no overdispersion. Or we can estimate it, and scale the standard errors from the regression model, thus accounting for overdispersion in the outcome. This is what the `quasipoisson` family does, and the coefficients it returns are identical to the Poisson's because the estimating equations are identical; only the standard errors change.

Other distributions can be affected by overdispersion (e.g., binomial distribution with $N > 1$). However, the Gaussian distribution and binomial distribution with $N = 1$ (i.e., a Bernoulli random variable) are not typically affected by overdispersion. In the Gaussian case, because the variance is estimated separately from the mean, overdispersion is not an issue. In the binomial/Bernoulli case, the scale parameter is set to 1 instead of estimating it. Other techniques are also available to handle overdispersion (e.g., using the negative binomial distribution instead, which allows the variance to exceed the mean for count outcome data).

The regression version of the negative binomial models $\log \mu$ as linear in the covariates, exactly as the Poisson does, and holds `size` fixed across observations. We can simulate from it with the same linear term as the Poisson example and `size = 2`, then fit three models to the result: the Poisson, the quasi-Poisson, and the negative binomial from MASS: `glm.nb()`, which estimates `size` alongside the coefficients:

```
set.seed(123)

n <- 5000

x <- rnorm(n)

y <- rnbinom(n, mu = exp(0.5 + log(1.5)*x), size = 2)
```

```
c(mean = mean(y), variance = var(y))
```

```
##      mean variance
## 1.769200 4.077947
```

```
fit_p <- glm(y ~ x, family = poisson("log"))
fit_q <- glm(y ~ x, family = quasipoisson("log"))
fit_nb <- MASS::glm.nb(y ~ x) # estimates omega outside the IRLS loop, allowing for overdispersion

rbind(poisson      = summary(fit_p)$coefficients["x", 1:2],
      quasipoisson = summary(fit_q)$coefficients["x", 1:2],
      `negative binomial` = summary(fit_nb)$coefficients["x", 1:2]) |>
  round(3)
```

```
##              Estimate Std. Error
## poisson           0.415      0.011
## quasipoisson       0.415      0.014
## negative binomial  0.424      0.015
```

```
round(summary(fit_q)$dispersion, 2)
```

```
## [1] 1.84
```

```
round(fit_nb$theta, 2)
```

```
## [1] 2.09
```

The variance of the simulated counts is more than twice their mean. All three slopes sit near $\log(1.5) = 0.41$, because all three models have the same mean model and the family only changes the weights. The Poisson's standard error is the one to distrust: it assumes a variance equal to the mean and is too small by about the square root of the estimated dispersion, 1.84. The quasi-Poisson repairs it by estimating that dispersion, and the negative binomial repairs it by estimating size, which comes back as 2.09 against the 2 we simulated, and keeps a full likelihood while doing so.

4.5 Gamma

The gamma distribution is defined by two parameters, a shape and a scale (or equivalently a rate, which is the inverse of the scale). It takes only positive values and is skewed to the right, with a mean of shape times scale and a variance of shape times scale squared, so that its standard deviation is proportional to its mean. That property makes it a natural choice for positive, right-skewed outcomes such as costs, concentrations, and lengths of stay. In R, `rgamma()` takes the shape and either the scale or the rate:

```
set.seed(123)

n <- 5

y <- rgamma(n, shape = 2, scale = 1)

y
```

```
## [1] 0.8920936 3.3118474 0.1443750 1.6625233 4.3358790
```

In a gamma regression model the mean is what depends on the covariates, usually through a log link, and the shape is held fixed across observations. To simulate from it, build the mean from a linear term and set each observation's scale to its mean divided by the common shape. The dispersion reported by `glm()` is one over that shape, which is the squared coefficient of variation of the outcome:

```
set.seed(123)

n <- 5000

x <- rnorm(n)

mu <- exp(1 + log(1.5)*x)

y <- rgamma(n, shape = 4, scale = mu / 4)
```

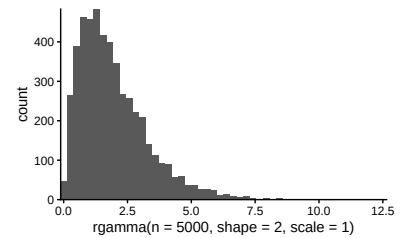


Figure 5: Histogram for the gamma distribution with shape 2 and scale 1 for 5000 simulated observations.

```
fit_gamma <- glm(y ~ x, family = Gamma("log"))

round(coef(fit_gamma), 3)
```

```
## (Intercept)          x
##      0.994      0.405
```

```
round(exp(coef(fit_gamma))["x"], 3)
```

```
##      x
## 1.5
```

```
round(summary(fit_gamma)$dispersion, 3)
```

```
## [1] 0.243
```

The slope returns $\log(1.5)$, so each unit of x multiplies the mean outcome by 1.5, and the estimated dispersion is 0.25, the inverse of the shape of 4 we simulated: the outcome's standard deviation is half its mean everywhere. The canonical link for the gamma is the inverse, which models $1/\mu$ as linear in the covariates. It is rarely used because its coefficients are on a scale nobody thinks in, and the log link, which keeps the mean positive, is the pairing seen in practice. It is a non-canonical pair that fits without complaint.

4.6 Inverse Gaussian

The inverse Gaussian distribution is a second choice for positive, right-skewed outcomes. Despite its name, it is not the distribution of one over a Gaussian variable; it is the distribution of the time a Brownian motion with positive drift takes to reach a fixed level, which is why it appears in models of durations and waiting times. It is defined by a mean μ and a shape parameter, and its variance is μ^3 divided by the shape, so the spread grows with the cube of the mean rather than the square. It is the member of the family to reach for when the variance climbs more steeply with the mean than the gamma allows. Base R has no generator for it; the `statmod` package provides one:

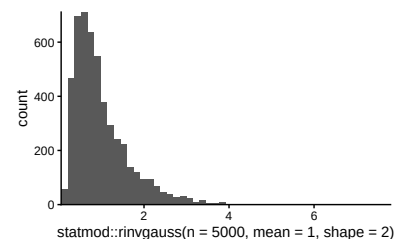


Figure 6: Histogram for the inverse Gaussian distribution with mean 1 and shape 2 for 5000 simulated observations.

```

set.seed(123)

n <- 5

y <- statmod::rinvgauss(n, mean = 1, shape = 2)

y

## [1] 1.4825554 0.8499473 0.3489367 1.0511153 0.9126631

```

The regression model has the same structure as the gamma's: the mean depends on the covariates through a link, the log link in practice, and the shape is held fixed, with the dispersion reported by `glm()` equal to one over the shape:

```

set.seed(123)

n <- 5000

x <- rnorm(n)

mu <- exp(1 + log(1.5)*x)

y <- statmod::rinvgauss(n, mean = mu, shape = 4)

fit_ig <- glm(y ~ x, family = inverse.gaussian("log"))

round(coef(fit_ig), 3)

## (Intercept)          x
##      1.015      0.397

round(exp(coef(fit_ig))["x"], 3)

##      x
## 1.488

```

```
round(summary(fit_ig)$dispersion, 3)
```

```
## [1] 0.247
```

Once again the slope returns $\log(1.5)$ and the dispersion returns one over the shape. The canonical link, $1/\mu^2$, is as rarely used as the gamma's inverse. The family is uncommon in epidemiology, but it completes the table, and it makes the point of the whole menu one last time: six distributions, one loop, and the choice among them is a choice about how the variance moves with the mean.

The table collects the menu for a single outcome and the contrasts each model returns.

Contrast	Family	Link	Canonical	Range risk	Model-based SE	Repair
Odds ratio	binomial	logit	yes	none	valid	
Risk ratio	binomial	log	no	mean above 1	valid	start values
Risk ratio	poisson	log	yes	mean above 1, silent	wrong	robust
Risk difference	binomial	identity	no	mean outside 0 to 1	valid	start values
Risk difference	gaussian	identity	yes	mean outside 0 to 1, silent	wrong	robust
Rate ratio	poisson + offset	log	yes	none	wrong if overdispersed	quasi or robust
Mean difference	gaussian	identity	yes	none	wrong if heteroscedastic	robust
Mean ratio	gaussian or Gamma	log	no / no	none	depends	robust

The menu for a single outcome. Each row is a family and link pair, the contrast its exposure coefficient estimates, whether the pair is canonical, where its fitted mean can leave its range, and whether its model-based standard errors can be trusted. “Robust” means the sandwich estimator of Week 7.

4.7 One Mean Model but Many Distributions

Every model in the menu was chosen by naming a family member of the exponential family. It is worth asking what that choice actually did to the coefficient. Mathematically, we can look at the score equation for the exponential family that the IRLS loop solves:

$$U_j = \sum_{i=1}^n \frac{(y_i - \mu_i)}{V(\mu_i)} \frac{\partial \mu_i}{\partial \eta_i} x_{ij}$$

From this equation, we see that the family really only enters in one place: the variance function in the denominator (which subsequently gets included as a weight in IRLS). The coefficient the loop solves for is a feature of the conditional mean. The distribution really only decides how much each observation is listened to (weighted). Here is an illustration that tries to make this concrete. Let's start with the crude model for the weight change indicator, with the log link and then the identity link, each fit under several families:

```
qsmk_coef <- function(formula, family, start = NULL) {
  fit <- glm(formula, data = nhefs, family = family, start = start)
  unname(coef(fit)["qsmk"])
}

crude <- wt_delta ~ qsmk
st <- c(log(mean(nhefs$wt_delta)), 0)

rr_crude <- c(binomial = qsmk_coef(crude, binomial("log"), st),
             poisson = qsmk_coef(crude, poisson("log"), st),
             gaussian = qsmk_coef(crude, gaussian("log"), st),
             quasibin = qsmk_coef(crude, quasibinomial("log"), st))

round(exp(rr_crude), 6)
```

```
## binomial poisson gaussian quasibin
## 1.275517 1.275517 1.275517 1.275517
```

```
rd_crude <- c(binomial = qsmk_coef(crude, binomial("identity")),
             gaussian = qsmk_coef(crude, gaussian("identity")),
             poisson = qsmk_coef(crude, poisson("identity")))

round(rd_crude, 6)
```

```
## binomial gaussian poisson
## 0.128638 0.128638 0.128638
```

These four families yield one risk ratio identical to six decimals. The three families yield one risk difference, which is also identical to six decimals. The

reason here is that the model is “saturated” (i.e., nonparametric). With a binary exposure and two parameters, the score equations are solved by setting each fitted mean equal to its observed cell mean, 0.467 among those who kept smoking and 0.596 among quitters, and no weight can change where that solution is. Every family with the log link returns $\log(0.596/0.467)$ and every family with the identity link returns $0.596 - 0.467$. The distribution never really kicks in to shape these numbers.

Now if we add the confounders, so that the model is no longer saturated, we get something different:

```
st <- c(log(mean(nhefs$wt_delta)), rep(0, 10))

rr_adj <- c(binomial = qsmk_coef(f_adj, binomial("log"), st),
           poisson = qsmk_coef(f_adj, poisson("log"), st),
           gaussian = qsmk_coef(f_adj, gaussian("log"), st))

round(exp(rr_adj), 4)

## binomial poisson gaussian
## 1.3332 1.3555 1.3475

rd_adj <- c(binomial = qsmk_coef(f_adj, binomial("identity")),
           gaussian = qsmk_coef(f_adj, gaussian("identity")))

round(rd_adj, 4)

## binomial gaussian
## 0.1573 0.1587
```

The risk ratios are 1.33, 1.36, and 1.35, and the risk differences 0.157 and 0.159. They are no longer identical, because the weights now differ from person to person and the model has no spare parameters to absorb the difference. But they are estimates of the same quantity, the adjusted risk ratio or risk difference that the mean model defines. If the mean model is right, all of them converge to it as the sample grows. The family now determines which estimate

is the most precise (i.e., the variance). The model whose variance function matches the outcome (the binomial, in this case) wins that contest.

If the mean model is wrong, the different weightings resolve the misspecification in slightly different ways, which is a reason to care about getting the mean model right and not a reason to prefer one family.

The same holds for a continuous outcome. Cholesterol regressed on quitting with a log link under four families, crude and then adjusted for age and sex:

```
chol_crude <- cholesterol ~ qsmk
chol_adj   <- cholesterol ~ qsmk + age + sex

mr <- sapply(list(crude = chol_crude, adjusted = chol_adj), function(f) {
  c(gaussian = qsmk_coef(f, gaussian("log")),
    Gamma     = qsmk_coef(f, Gamma("log")),
    quasipois = qsmk_coef(f, quasipoisson("log")),
    invgauss  = qsmk_coef(f, inverse.gaussian("log")))
})

round(exp(mr), 4)
```

```
##           crude adjusted
## gaussian  1.0196   1.0038
## Gamma     1.0196   1.0052
## quasipois 1.0196   1.0045
## invgauss  1.0196   1.0058
```

Here, we get one mean ratio exactly, four times in the crude column, and we get four estimates between 1.004 and 1.006 in the adjusted one. A Poisson family for an outcome measured in milligrams per deciliter is nonsense as a distribution. Yet still, the model estimates a sensible mean for this outcome.

Where the family does matter, once again, is in the standard error. Because $(X'WX)^{-1}$ carries the variance function into every model-based standard error. For the adjusted log-link models of the binary outcome, we get:

```

mod_bl <- glm(f_adj, data = nhfs, family = binomial("log"), start = st)
mod_pl <- glm(f_adj, data = nhfs, family = poisson("log"))

rob_pl <- lmtest::coeftest(mod_pl, vcov = sandwich::sandwich(mod_pl)) # using empirical, or robust, or

se_tab <- rbind(
  `binomial, model-based` = summary(mod_bl)$coefficients["qsmk", 1:2],
  `poisson, model-based`  = summary(mod_pl)$coefficients["qsmk", 1:2],
  `poisson, robust`      = rob_pl["qsmk", 1:2])

round(se_tab, 4)

```

##	Estimate	Std. Error
## binomial, model-based	0.2876	0.0501
## poisson, model-based	0.3041	0.0795
## poisson, robust	0.3041	0.0520

The Poisson's model-based standard error is sixty percent too large, because its variance function is μ , where a binary outcome's variance is $\mu(1 - \mu)$. The robust standard error here, which drops the variance function and uses the observed residuals instead, brings it back to within four percent of the binomial's.

So, this is an illustration of the division of labor in a GLM, so to speak. The link and the linear predictor decide what the coefficient means, through the conditional expectation function. The values of the coefficients are influenced by the variance of the distribution (via the weights in IRLS), but this influence tends to be small. The variance function, however, completely shapes the model-based standard errors. When it is wrong, the robust variance estimator can repair the damage, without affecting the coefficient values at all.

Thus, choosing the family is really a modeling choice about the variance.

5 Conclusion

This lecture focused on GLMs from a specific perspective. Every generalized linear model is iteratively reweighted least squares with a variance function and a link plugged in. The exponential family is the structure that makes this

Model	Family, link	Contrast	Variance function	Where it breaks	Repair
Linear	gaussian, identity	mean difference	constant	heteroscedasticity	robust SE
Logistic	binomial, logit	odds ratio	$\mu(1 - \mu)$	separation	recode or penalize
Log-binomial	binomial, log	risk ratio	$\mu(1 - \mu)$	fitted risk above 1	start values or Poisson route
Poisson	poisson, log	rate or risk ratio	μ	overdispersion	quasi, NB, or robust SE
Gamma	Gamma, log	mean ratio	μ^2	zeros in the outcome	different family

work, because in that family a single function b determines both the mean and the variance function, and the loop needs nothing else. The canonical link is the case where the slope of the link function is the reciprocal of the variance function (the mean responds to the linear predictor at exactly the rate set by the variance function, so that the two cancel in the score equation), which buys us some nice properties.

However, we may want to forego these properties to use GLMs in another way: as a tool to estimate conditional mean functions on a particular scale of interest. If needed, we can adjust the variances of the model by accounting for dispersion parameters, or by correcting the variance using the robust (sandwich) variance estimator. Ultimately, this perspective allows us to use the GLM framework in a much more practical “swiss army knife” approach.

References

- A. Colin Cameron and Pravin K. Trivedi. *Regression Analysis of Count Data*. Cambridge University Press, Cambridge, 2nd edition, 2013.
- George Casella and Roger L. Berger. *Statistical Inference*. Duxbury Press, Pacific Grove, CA, 2nd edition, 2002.
- Annette J. Dobson. *An Introduction to Generalized Linear Models*. Texts in Statistical Science. Chapman & Hall/CRC, Boca Raton, FL, 2nd edition, 2002.
- David Firth. Bias reduction of maximum likelihood estimates. *Biometrika*, 80(1):27–38, 1993.
- Lawrence M Leemis, , and Jacquelyn T McQueston. Univariate distribution relationships. *The American Statistician*, 62(1):45–53, 2025/05/27/16:12:02 2008. DOI: 10.1198/000313008X270448. URL <https://doi.org/10.1198/000313008X270448>.
- J O Lloyd-Smith, S J Schreiber, P E Kopp, and W M Getz. Superspreading and the effect of individual variation on disease emergence. *Nature*, 438(7066): 355–359, Nov 2005. ISSN 1476-4687 (Electronic); 0028-0836 (Print); 0028-0836 (Linking). DOI: 10.1038/nature04153.
- Ian C. Marschner and Alexandra C. Gillett. Relative risk regression: Reliable and flexible methods for log-binomial models. *Biostatistics*, 13(1):179–192, 2012.
- Peter McCullagh and John A. Nelder. *Generalized Linear Models*. Chapman and Hall, London, 2nd edition, 1989.
- J. A. Nelder and R. W. M. Wedderburn. Generalized linear models. *JRSS-A*, 135(3):370–384, 1972.
- G. A. F. Seber and C. J. Wild. *Nonlinear regression*. Wiley, New York, 1989.
- Guangyong Zou. A modified poisson regression approach to prospective studies with binary data. *Am J Epidemiol*, 159(7):702–706, Apr 2004.